JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Long

java.lang.Object
    java.lang.Number
        java.lang.Long

All Implemented Interfaces:

`Serializable, Comparable<Long>`

```
public final class Long
extends Number
implements Comparable<Long>
```

The `Long` class wraps a value of the primitive type `long` in an object. An object of type `Long` contains a single field whose type is `long`.

In addition, this class provides several methods for converting a `long` to a `String` and a `String` to a `long`, as well as other constants and methods useful when dealing with a `long`.

Implementation note: The implementations of the "bit twiddling" methods (such as `highestOneBit` and `numberOfTrailingZeros`) are based on material from Henry S. Warren, Jr.'s *Hacker's Delight*, (Addison Wesley, 2002).

Since:

`JDK1.0`

See Also:

`Serialized Form`

Field Summary

Fields

Modifier and Type	Field	Description
static int	BYTES	The number of bytes used to represent a long value in two's complement binary form.
static long	MAX_VALUE	A constant holding the maximum value a long can have, 2^{63}-1.
static long	MIN_VALUE	A constant holding the minimum value a long can have, -2^{63}.
static int	SIZE	The number of bits used to represent a long value in two's complement binary form.
static Class<Long>	TYPE	The Class instance representing the primitive type long.

Constructor Summary

Constructors

Constructor	Description
Long(long value)	Constructs a newly allocated Long object that represents the specified long argument.
Long(String s)	Constructs a newly allocated Long object that represents the long value indicated by the String parameter.

Method Summary

Concrete Methods **Instance Method**

Modifier and Type	Method	Description
static int	bitCount(long i)	Returns the number of one-bits in the two's complement binary representation of the specified long value.

byte	**byteValue**()	Returns the value of this Long as a byte after a narrowing primitive conversion.
static int	**compare**(long x, long y)	Compares two long values numerically.
int	**compareTo**(**Long** anotherLong)	Compares two Long objects numerically.
static int	**compareUnsigned**(long x, long y)	Compares two long values numerically treating the values as unsigned.
static **Long**	**decode**(**String** nm)	Decodes a String into a Long.
static long	**divideUnsigned**(long dividend, long divisor)	Returns the unsigned quotient of dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.
double	**doubleValue**()	Returns the value of this Long as a double after a widening primitive conversion.
boolean	**equals**(**Object** obj)	Compares this object to the specified object.
float	**floatValue**()	Returns the value of this Long as a float after a

		widening primitive conversion.
static **Long**	**getLong**(**String** nm)	Determines the long value of the system property with the specified name.
static **Long**	**getLong**(**String** nm, long val)	Determines the long value of the system property with the specified name.
static **Long**	**getLong**(**String** nm, **Long** val)	Returns the long value of the system property with the specified name.
int	**hashCode**()	Returns a hash code for this Long.
static int	**hashCode**(long value)	Returns a hash code for a long value; compatible with Long.hashCode().
static long	**highestOneBit**(long i)	Returns a long value with at most a single one-bit, in the position of the highest-order ("leftmost") one-bit in the specified long value.
int	**intValue**()	Returns the value of this Long as an int after a narrowing primitive conversion.
long	**longValue**()	

		Returns the value of this Long as a long value.
static long	**lowestOneBit**(long i)	Returns a long value with at most a single one-bit, in the position of the lowest-order ("rightmost") one-bit in the specified long value.
static long	**max**(long a, long b)	Returns the greater of two long values as if by calling **Math.max**.
static long	**min**(long a, long b)	Returns the smaller of two long values as if by calling **Math.min**.
static int	**numberOfLeadingZeros**(long i)	Returns the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified long value.
static int	**numberOfTrailingZeros**(long i)	Returns the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of

		the specified `long` value.
static long	**parseLong**(**String** s)	Parses the string argument as a signed decimal `long`.
static long	**parseLong**(**String** s, int radix)	Parses the string argument as a signed `long` in the radix specified by the second argument.
static long	**parseUnsignedLong**(**String** s)	Parses the string argument as an unsigned decimal `long`.
static long	**parseUnsignedLong**(**String** s, int radix)	Parses the string argument as an unsigned `long` in the radix specified by the second argument.
static long	**remainderUnsigned**(long dividend, long divisor)	Returns the unsigned remainder from dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.
static long	**reverse**(long i)	Returns the value obtained by reversing the order of the bits in the two's complement binary representation of the specified `long` value.
static long	**reverseBytes**(long i)	

		Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified `long` value.
static long	**rotateLeft**(long i, int distance)	Returns the value obtained by rotating the two's complement binary representation of the specified `long` value left by the specified number of bits.
static long	**rotateRight**(long i, int distance)	Returns the value obtained by rotating the two's complement binary representation of the specified `long` value right by the specified number of bits.
short	**shortValue**()	Returns the value of this `Long` as a `short` after a narrowing primitive conversion.
static int	**signum**(long i)	Returns the signum function of the specified `long` value.
static long	**sum**(long a, long b)	Adds two `long` values together as per the + operator.

static **String**	**toBinaryString**(long i)	Returns a string representation of the long argument as an unsigned integer in base 2.
static **String**	**toHexString**(long i)	Returns a string representation of the long argument as an unsigned integer in base 16.
static **String**	**toOctalString**(long i)	Returns a string representation of the long argument as an unsigned integer in base 8.
String	**toString**()	Returns a String object representing this Long's value.
static **String**	**toString**(long i)	Returns a String object representing the specified long.
static **String**	**toString**(long i, int radix)	Returns a string representation of the first argument in the radix specified by the second argument.
static **String**	**toUnsignedString**(long i)	Returns a string representation of the argument as an unsigned decimal value.
static **String**	**toUnsignedString**(long i, int radix)	Returns a string representation of the first argument as an unsigned integer value in

		the radix specified by the second argument.
static **Long**	**valueOf**(long l)	Returns a Long instance representing the specified long value.
static **Long**	**valueOf**(**String** s)	Returns a Long object holding the value of the specified String.
static **Long**	**valueOf**(**String** s, int radix)	Returns a Long object holding the value extracted from the specified String when parsed with the radix given by the second argument.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

MIN_VALUE

```
@Native
public static final long MIN_VALUE
```

A constant holding the minimum value a long can have, -2^{63}.

See Also:

Constant Field Values

MAX_VALUE

```
@Native
public static final long MAX_VALUE
```

A constant holding the maximum value a `long` can have, 2^{63}-1.

See Also:

Constant Field Values

TYPE

```
public static final Class<Long> TYPE
```

The `Class` instance representing the primitive type `long`.

Since:

JDK1.1

SIZE

```
@Native
public static final int SIZE
```

The number of bits used to represent a `long` value in two's complement binary form.

Since:

1.5

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `long` value in two's complement binary form.

Since:

1.8

See Also:

Constant Field Values

Constructor Detail

Long

```
public Long(long value)
```

Constructs a newly allocated `Long` object that represents the specified `long` argument.

Parameters:

`value - the value to be represented by the Long object.`

Long

```
public Long(String s)
     throws NumberFormatException
```

Constructs a newly allocated `Long` object that represents the `long` value indicated by the `String` parameter. The string is converted to a `long` value in exactly the manner used by the `parseLong` method for radix 10.

Parameters:

`s - the String to be converted to a Long.`

Throws:

`NumberFormatException - if the String does not contain a parsable long.`

See Also:

`parseLong(java.lang.String, int)`

Method Detail

toString

```
public static String toString(long i,
                              int radix)
```

Returns a string representation of the first argument in the radix specified by the second argument.

If the radix is smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`, then the radix `10` is used instead.

If the first argument is negative, the first element of the result is the ASCII minus sign `'-'` (`'\u002d'`). If the first argument is not negative, no sign character appears in the result.

The remaining characters of the result represent the magnitude of the first argument. If the magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the magnitude will not be the zero character. The following ASCII characters are used as digits:

```
0123456789abcdefghijklmnopqrstuvwxyz
```

These are `'\u0030'` through `'\u0039'` and `'\u0061'` through `'\u007a'`. If `radix` is *N*, then the first *N* of these characters are used as radix-*N* digits in the order shown. Thus, the digits for hexadecimal (radix 16) are `0123456789abcdef`. If uppercase letters are desired, the `String.toUpperCase()` method may be called on the result:

```
Long.toString(n, 16).toUpperCase()
```

Parameters:

`i - a long to be converted to a string.`

`radix - the radix to use in the string representation.`

Returns:

`a string representation of the argument in the specified radix.`

See Also:

`Character.MAX_RADIX`, `Character.MIN_RADIX`

toUnsignedString

```
public static String toUnsignedString(long i,
                                      int radix)
```

Returns a string representation of the first argument as an unsigned integer value in the radix specified by the second argument.

If the radix is smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`, then the radix `10` is used instead.

Note that since the first argument is treated as an unsigned value, no leading sign character is printed.

If the magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the magnitude will not be the zero character.

The behavior of radixes and the characters used as digits are the same as `toString`.

Parameters:

`i - an integer to be converted to an unsigned string.`

`radix - the radix to use in the string representation.`

Returns:

`an unsigned string representation of the argument in the specified radix.`

Since:

`1.8`

See Also:

`toString(long, int)`

toHexString

```
public static String toHexString(long i)
```

Returns a string representation of the `long` argument as an unsigned integer in base 16.

The unsigned `long` value is the argument plus 2^{64} if the argument is negative; otherwise, it is equal to the argument. This value is converted to a string of ASCII digits in hexadecimal (base 16) with no extra leading `0`s.

The value of the argument can be recovered from the returned string `s` by calling `Long.parseUnsignedLong(s, 16)`.

If the unsigned magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The following characters are used as hexadecimal digits:

```
0123456789abcdef
```

These are the characters `'\u0030'` through `'\u0039'` and `'\u0061'` through `'\u0066'`. If uppercase letters are desired, the `String.toUpperCase()` method may be called on the result:

```
Long.toHexString(n).toUpperCase()
```

Parameters:

`i - a long to be converted to a string.`

Returns:

`the string representation of the unsigned long value represented by the argument in hexadecimal (base 16).`

Since:

`JDK 1.0.2`

See Also:

parseUnsignedLong(String, int), toUnsignedString(long, int)

toOctalString

```
public static String toOctalString(long i)
```

Returns a string representation of the long argument as an unsigned integer in base 8.

The unsigned long value is the argument plus 2^{64} if the argument is negative; otherwise, it is equal to the argument. This value is converted to a string of ASCII digits in octal (base 8) with no extra leading 0s.

The value of the argument can be recovered from the returned string s by calling Long.parseUnsignedLong(s, 8).

If the unsigned magnitude is zero, it is represented by a single zero character '0' ('\u0030'); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The following characters are used as octal digits:

```
01234567
```

These are the characters '\u0030' through '\u0037'.

Parameters:

i - a long to be converted to a string.

Returns:

the string representation of the unsigned long value represented by the argument in octal (base 8).

Since:

JDK 1.0.2

See Also:

parseUnsignedLong(String, int), toUnsignedString(long, int)

toBinaryString

```
public static String toBinaryString(long i)
```

Returns a string representation of the long argument as an unsigned integer in base 2.

The unsigned long value is the argument plus 2^{64} if the argument is negative; otherwise, it is equal to the argument. This value is converted to a string of ASCII digits in binary (base 2) with no extra leading 0s.

The value of the argument can be recovered from the returned string `s` by calling `Long.parseUnsignedLong(s, 2)`.

If the unsigned magnitude is zero, it is represented by a single zero character `'0'` (`'\u0030'`); otherwise, the first character of the representation of the unsigned magnitude will not be the zero character. The characters `'0'` (`'\u0030'`) and `'1'` (`'\u0031'`) are used as binary digits.

Parameters:

`i - a long to be converted to a string.`

Returns:

`the string representation of the unsigned long value represented by the argument in binary (base 2).`

Since:

`JDK 1.0.2`

See Also:

`parseUnsignedLong(String, int)`, `toUnsignedString(long, int)`

toString

`public static String toString(long i)`

Returns a `String` object representing the specified `long`. The argument is converted to signed decimal representation and returned as a string, exactly as if the argument and the radix 10 were given as arguments to the `toString(long, int)` method.

Parameters:

`i - a long to be converted.`

Returns:

`a string representation of the argument in base 10.`

toUnsignedString

`public static String toUnsignedString(long i)`

Returns a string representation of the argument as an unsigned decimal value. The argument is converted to unsigned decimal representation and returned as a string exactly as if the argument and radix 10 were given as arguments to the `toUnsignedString(long, int)` method.

Parameters:

`i - an integer to be converted to an unsigned string.`

Returns:

`an unsigned string representation of the argument.`

Since:

1.8

See Also:

`toUnsignedString(long, int)`

parseLong

```
public static long parseLong(String s,
                             int radix)
                      throws NumberFormatException
```

Parses the string argument as a signed `long` in the radix specified by the second argument. The characters in the string must all be digits of the specified radix (as determined by whether `Character.digit(char, int)` returns a nonnegative value), except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `long` value is returned.

Note that neither the character `L` (`'\u004C'`) nor `l` (`'\u006C'`) is permitted to appear at the end of the string as a type indicator, as would be permitted in Java programming language source code - except that either `L` or `l` may appear as a digit for a radix greater than or equal to 22.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The `radix` is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.
- Any character of the string is not a digit of the specified radix, except that the first character may be a minus sign `'-'` (`'\u002d'`) or plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is not a value of type `long`.

Examples:

```
parseLong("0", 10) returns 0L
parseLong("473", 10) returns 473L
parseLong("+42", 10) returns 42L
parseLong("-0", 10) returns 0L
parseLong("-FF", 16) returns -255L
parseLong("1100110", 2) returns 102L
parseLong("99", 8) throws a NumberFormatException
parseLong("Hazelnut", 10) throws a NumberFormatException
parseLong("Hazelnut", 36) returns 1356099454469L
```

Parameters:

`s - the String containing the long representation to be parsed.`

`radix - the radix to be used while parsing s.`

Returns:

`the long represented by the string argument in the specified radix.`

Throws:

`NumberFormatException - if the string does not contain a parsable long.`

parseLong

```
public static long parseLong(String s)
                      throws NumberFormatException
```

Parses the string argument as a signed decimal `long`. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign `'-'` (`\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `long` value is returned, exactly as if the argument and the radix `10` were given as arguments to the `parseLong(java.lang.String, int)` method.

Note that neither the character `L` (`'\u004C'`) nor `l` (`'\u006C'`) is permitted to appear at the end of the string as a type indicator, as would be permitted in Java programming language source code.

Parameters:

`s - a String containing the long representation to be parsed`

Returns:

`the long represented by the argument in decimal.`

Throws:

`NumberFormatException - if the string does not contain a parsable long.`

parseUnsignedLong

```
public static long parseUnsignedLong(String s,
                                     int radix)
                              throws NumberFormatException
```

Parses the string argument as an unsigned `long` in the radix specified by the second argument. An unsigned integer maps the values usually associated with negative numbers to positive numbers larger than `MAX_VALUE`. The characters in the string must all be digits of the specified radix (as

determined by whether `Character.digit(char, int)` returns a nonnegative value), except that the first character may be an ASCII plus sign `'+'` (`'\u002B'`). The resulting integer value is returned.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The radix is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.
- Any character of the string is not a digit of the specified radix, except that the first character may be a plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is larger than the largest unsigned `long`, 2^{64}-1.

Parameters:

`s - the String containing the unsigned integer representation to be parsed`

`radix - the radix to be used while parsing s.`

Returns:

`the unsigned long represented by the string argument in the specified radix.`

Throws:

`NumberFormatException - if the String does not contain a parsable long.`

Since:

`1.8`

parseUnsignedLong

```
public static long parseUnsignedLong(String s)
                              throws NumberFormatException
```

Parses the string argument as an unsigned decimal `long`. The characters in the string must all be decimal digits, except that the first character may be an an ASCII plus sign `'+'` (`'\u002B'`). The resulting integer value is returned, exactly as if the argument and the radix 10 were given as arguments to the `parseUnsignedLong(java.lang.String, int)` method.

Parameters:

`s - a String containing the unsigned long representation to be parsed`

Returns:

`the unsigned long value represented by the decimal string argument`

Throws:

NumberFormatException - if the string does not contain a parsable unsigned integer.

Since:

1.8

valueOf

```
public static Long valueOf(String s,
                           int radix)
                    throws NumberFormatException
```

Returns a Long object holding the value extracted from the specified String when parsed with the radix given by the second argument. The first argument is interpreted as representing a signed long in the radix specified by the second argument, exactly as if the arguments were given to the parseLong(java.lang.String, int) method. The result is a Long object that represents the long value specified by the string.

In other words, this method returns a Long object equal to the value of:

```
new Long(Long.parseLong(s, radix))
```

Parameters:

s - the string to be parsed

radix - the radix to be used in interpreting s

Returns:

a Long object holding the value represented by the string argument in the specified radix.

Throws:

NumberFormatException - If the String does not contain a parsable long.

valueOf

```
public static Long valueOf(String s)
                    throws NumberFormatException
```

Returns a Long object holding the value of the specified String. The argument is interpreted as representing a signed decimal long, exactly as if the argument were given to the parseLong(java.lang.String) method. The result is a Long object that represents the integer value specified by the string.

In other words, this method returns a Long object equal to the value of:

```
new Long(Long.parseLong(s))
```

Parameters:

`s - the string to be parsed.`

Returns:

`a Long object holding the value represented by the string argument.`

Throws:

`NumberFormatException - If the string cannot be parsed as a long.`

valueOf

```
public static Long valueOf(long l)
```

Returns a `Long` instance representing the specified `long` value. If a new `Long` instance is not required, this method should generally be used in preference to the constructor `Long(long)`, as this method is likely to yield significantly better space and time performance by caching frequently requested values. Note that unlike the corresponding method in the `Integer` class, this method is *not* required to cache values within a particular range.

Parameters:

`l - a long value.`

Returns:

`a Long instance representing l.`

Since:

`1.5`

decode

```
public static Long decode(String nm)
                  throws NumberFormatException
```

Decodes a `String` into a `Long`. Accepts decimal, hexadecimal, and octal numbers given by the following grammar:

DecodableString:

$Sign_{opt}$ *DecimalNumeral*

$Sign_{opt}$ 0x *HexDigits*

$Sign_{opt}$ 0X *HexDigits*

$Sign_{opt}$ # *HexDigits*

$Sign_{opt}$ 0 *OctalDigits*

Sign:

-

+

DecimalNumeral, *HexDigits*, and *OctalDigits* are as defined in section 3.10.1 of *The Java™ Language Specification*, except that underscores are not accepted between digits.

The sequence of characters following an optional sign and/or radix specifier ("0x", "0X", "#", or leading zero) is parsed as by the `Long.parseLong` method with the indicated radix (10, 16, or 8). This sequence of characters must represent a positive value or a `NumberFormatException` will be thrown. The result is negated if first character of the specified `String` is the minus sign. No whitespace characters are permitted in the `String`.

Parameters:

`nm - the String to decode.`

Returns:

`a Long object holding the long value represented by nm`

Throws:

`NumberFormatException - if the String does not contain a parsable long.`

Since:

`1.2`

See Also:

`parseLong(String, int)`

byteValue

```
public byte byteValue()
```

Returns the value of this `Long` as a `byte` after a narrowing primitive conversion.

Overrides:

`byteValue in class Number`

Returns:

`the numeric value represented by this object after conversion to type byte.`

See *The Java™ Language Specification*:

`5.1.3 Narrowing Primitive Conversions`

shortValue

```
public short shortValue()
```

Returns the value of this Long as a short after a narrowing primitive conversion.

Overrides:

shortValue in class Number

Returns:

the numeric value represented by this object after conversion to type short.

See *The Java™ Language Specification*:

5.1.3 Narrowing Primitive Conversions

intValue

```
public int intValue()
```

Returns the value of this Long as an int after a narrowing primitive conversion.

Specified by:

intValue in class Number

Returns:

the numeric value represented by this object after conversion to type int.

See *The Java™ Language Specification*:

5.1.3 Narrowing Primitive Conversions

longValue

```
public long longValue()
```

Returns the value of this Long as a long value.

Specified by:

longValue in class Number

Returns:

the numeric value represented by this object after conversion to type long.

floatValue

```
public float floatValue()
```

Returns the value of this `Long` as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type float.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

doubleValue

```
public double doubleValue()
```

Returns the value of this `Long` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type double.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

toString

```
public String toString()
```

Returns a `String` object representing this `Long`'s value. The value is converted to signed decimal representation and returned as a string, exactly as if the `long` value were given as an argument to the `toString(long)` method.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the value of this object in base 10.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Long`. The result is the exclusive OR of the two halves of the primitive `long` value held by this `Long` object. That is, the hashcode is the value of the expression:

```
(int)(this.longValue()^(this.longValue()>>>32))
```

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

hashCode

```
public static int hashCode(long value)
```

Returns a hash code for a `long` value; compatible with `Long.hashCode()`.

Parameters:

`value` - the value to hash

Returns:

a hash code value for a long value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object to the specified object. The result is `true` if and only if the argument is not `null` and is a `Long` object that contains the same `long` value as this object.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to compare with.

Returns:

true if the objects are the same; false otherwise.

See Also:

Object.hashCode(), HashMap

getLong

```
public static Long getLong(String nm)
```

Determines the long value of the system property with the specified name.

The first argument is treated as the name of a system property. System properties are accessible through the System.getProperty(java.lang.String) method. The string value of this property is then interpreted as a long value using the grammar supported by decode and a Long object representing this value is returned.

If there is no property with the specified name, if the specified name is empty or null, or if the property does not have the correct numeric format, then null is returned.

In other words, this method returns a Long object equal to the value of:

```
getLong(nm, null)
```

Parameters:

nm - property name.

Returns:

the Long value of the property.

Throws:

SecurityException - for the same reasons as System.getProperty

See Also:

System.getProperty(java.lang.String), System.getProperty(java.lang.String, java.lang.String)

getLong

```
public static Long getLong(String nm,
                           long val)
```

Determines the long value of the system property with the specified name.

The first argument is treated as the name of a system property. System properties are accessible through the System.getProperty(java.lang.String) method. The string value of this property is then interpreted as a long value using the grammar supported by decode and a Long object representing this value is returned.

The second argument is the default value. A Long object that represents the value of the second argument is returned if there is no property of the specified name, if the property does not have the correct numeric format, or if the specified name is empty or null.

In other words, this method returns a Long object equal to the value of:

```
getLong(nm, new Long(val))
```

but in practice it may be implemented in a manner such as:

```
Long result = getLong(nm, null);
return (result == null) ? new Long(val) : result;
```

to avoid the unnecessary allocation of a Long object when the default value is not needed.

Parameters:

nm - property name.

val - default value.

Returns:

the Long value of the property.

Throws:

SecurityException - for the same reasons as System.getProperty

See Also:

System.getProperty(java.lang.String), System.getProperty(java.lang.String, java.lang.String)

getLong

```
public static Long getLong(String nm,
                           Long val)
```

Returns the long value of the system property with the specified name. The first argument is treated as the name of a system property. System properties are accessible through the System.getProperty(java.lang.String) method. The string value of this property is then interpreted as a long value, as per the decode method, and a Long object representing this value is returned; in summary:

- If the property value begins with the two ASCII characters 0x or the ASCII character #, not followed by a minus sign, then the rest of it is parsed as a hexadecimal integer exactly as for the method valueOf(java.lang.String, int) with radix 16.
- If the property value begins with the ASCII character 0 followed by another character, it is parsed as an octal integer exactly as by the method valueOf(java.lang.String, int) with radix 8.

- Otherwise the property value is parsed as a decimal integer exactly as by the method `valueOf(java.lang.String, int)` with radix 10.

Note that, in every case, neither `L` (`'\u004C'`) nor `l` (`'\u006C'`) is permitted to appear at the end of the property value as a type indicator, as would be permitted in Java programming language source code.

The second argument is the default value. The default value is returned if there is no property of the specified name, if the property does not have the correct numeric format, or if the specified name is empty or `null`.

Parameters:

`nm - property name.`

`val - default value.`

Returns:

`the Long value of the property.`

Throws:

`SecurityException - for the same reasons as System.getProperty`

See Also:

`System.getProperty(java.lang.String)`, `System.getProperty(java.lang.String, java.lang.String)`

compareTo

```
public int compareTo(Long anotherLong)
```

Compares two `Long` objects numerically.

Specified by:

`compareTo in interface Comparable<Long>`

Parameters:

`anotherLong - the Long to be compared.`

Returns:

`the value 0 if this Long is equal to the argument Long; a value less than 0 if this Long is numerically less than the argument Long; and a value greater than 0 if this Long is numerically greater than the argument Long (signed comparison).`

Since:

`1.2`

compare

```
public static int compare(long x,
                          long y)
```

Compares two `long` values numerically. The value returned is identical to what would be returned by:

```
    Long.valueOf(x).compareTo(Long.valueOf(y))
```

Parameters:

`x - the first long to compare`

`y - the second long to compare`

Returns:

`the value 0 if x == y; a value less than 0 if x < y; and a value greater than 0 if x > y`

Since:

`1.7`

compareUnsigned

```
public static int compareUnsigned(long x,
                                  long y)
```

Compares two `long` values numerically treating the values as unsigned.

Parameters:

`x - the first long to compare`

`y - the second long to compare`

Returns:

`the value 0 if x == y; a value less than 0 if x < y as unsigned values; and a value greater than 0 if x > y as unsigned values`

Since:

`1.8`

divideUnsigned

```
public static long divideUnsigned(long dividend,
                                  long divisor)
```

Returns the unsigned quotient of dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.

Note that in two's complement arithmetic, the three other basic arithmetic operations of add, subtract, and multiply are bit-wise identical if the two operands are regarded as both being signed or both being unsigned. Therefore separate `addUnsigned`, etc. methods are not provided.

Parameters:

`dividend - the value to be divided`

`divisor - the value doing the dividing`

Returns:

`the unsigned quotient of the first argument divided by the second argument`

Since:

`1.8`

See Also:

`remainderUnsigned(long, long)`

remainderUnsigned

```
public static long remainderUnsigned(long dividend,
                                     long divisor)
```

Returns the unsigned remainder from dividing the first argument by the second where each argument and the result is interpreted as an unsigned value.

Parameters:

`dividend - the value to be divided`

`divisor - the value doing the dividing`

Returns:

`the unsigned remainder of the first argument divided by the second argument`

Since:

`1.8`

See Also:

`divideUnsigned(long, long)`

highestOneBit

```
public static long highestOneBit(long i)
```

Returns a `long` value with at most a single one-bit, in the position of the highest-order ("leftmost") one-bit in the specified `long` value. Returns zero if

the specified value has no one-bits in its two's complement binary representation, that is, if it is equal to zero.

Parameters:

`i - the value whose highest one bit is to be computed`

Returns:

`a long value with a single one-bit, in the position of the highest-order one-bit in the specified value, or zero if the specified value is itself equal to zero.`

Since:

`1.5`

lowestOneBit

```
public static long lowestOneBit(long i)
```

Returns a `long` value with at most a single one-bit, in the position of the lowest-order ("rightmost") one-bit in the specified `long` value. Returns zero if the specified value has no one-bits in its two's complement binary representation, that is, if it is equal to zero.

Parameters:

`i - the value whose lowest one bit is to be computed`

Returns:

`a long value with a single one-bit, in the position of the lowest-order one-bit in the specified value, or zero if the specified value is itself equal to zero.`

Since:

`1.5`

numberOfLeadingZeros

```
public static int numberOfLeadingZeros(long i)
```

Returns the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified `long` value. Returns 64 if the specified value has no one-bits in its two's complement representation, in other words if it is equal to zero.

Note that this method is closely related to the logarithm base 2. For all positive `long` values x:

- floor($\log_2$(x)) = `63 - numberOfLeadingZeros(x)`
- ceil($\log_2$(x)) = `64 - numberOfLeadingZeros(x - 1)`

Parameters:

i - the value whose number of leading zeros is to be computed

Returns:

the number of zero bits preceding the highest-order ("leftmost") one-bit in the two's complement binary representation of the specified long value, or 64 if the value is equal to zero.

Since:

1.5

numberOfTrailingZeros

```
public static int numberOfTrailingZeros(long i)
```

Returns the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of the specified `long` value. Returns 64 if the specified value has no one-bits in its two's complement representation, in other words if it is equal to zero.

Parameters:

i - the value whose number of trailing zeros is to be computed

Returns:

the number of zero bits following the lowest-order ("rightmost") one-bit in the two's complement binary representation of the specified long value, or 64 if the value is equal to zero.

Since:

1.5

bitCount

```
public static int bitCount(long i)
```

Returns the number of one-bits in the two's complement binary representation of the specified `long` value. This function is sometimes referred to as the *population count*.

Parameters:

i - the value whose bits are to be counted

Returns:

the number of one-bits in the two's complement binary representation of the specified long value.

Since:

1.5

rotateLeft

```
public static long rotateLeft(long i,
                              int distance)
```

Returns the value obtained by rotating the two's complement binary representation of the specified `long` value left by the specified number of bits. (Bits shifted out of the left hand, or high-order, side reenter on the right, or low-order.)

Note
that left rotation with a negative distance is equivalent to right rotation: `rotateLeft(val, -distance) == rotateRight(val, distance)`. Note also that rotation by any multiple of 64 is a no-op, so all but the last six bits of the rotation distance can be ignored, even if the distance is negative: `rotateLeft(val, distance) == rotateLeft(val, distance & 0x3F)`.

Parameters:

`i - the value whose bits are to be rotated left`

`distance - the number of bit positions to rotate left`

Returns:

`the value obtained by rotating the two's complement binary representation of the specified long value left by the specified number of bits.`

Since:

`1.5`

rotateRight

```
public static long rotateRight(long i,
                               int distance)
```

Returns the value obtained by rotating the two's complement binary representation of the specified `long` value right by the specified number of bits. (Bits shifted out of the right hand, or low-order, side reenter on the left, or high-order.)

Note
that right rotation with a negative distance is equivalent to left rotation: `rotateRight(val, -distance) == rotateLeft(val, distance)`. Note also that rotation by any multiple of 64 is a no-op, so all but the last six bits of the rotation distance can be ignored, even if the distance is negative: `rotateRight(val, distance) == rotateRight(val, distance & 0x3F)`.

Parameters:

`i - the value whose bits are to be rotated right`

distance - the number of bit positions to rotate right

Returns:

the value obtained by rotating the two's complement binary representation of the specified long value right by the specified number of bits.

Since:

1.5

reverse

```
public static long reverse(long i)
```

Returns the value obtained by reversing the order of the bits in the two's complement binary representation of the specified `long` value.

Parameters:

i - the value to be reversed

Returns:

the
value obtained by reversing order of the bits in the specified long value.

Since:

1.5

signum

```
public static int signum(long i)
```

Returns the signum function of the specified `long` value. (The return value is -1 if the specified value is negative; 0 if the specified value is zero; and 1 if the specified value is positive.)

Parameters:

i - the value whose signum is to be computed

Returns:

the signum function of the specified long value.

Since:

1.5

reverseBytes

```
public static long reverseBytes(long i)
```

Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified `long` value.

Parameters:

`i - the value whose bytes are to be reversed`

Returns:

`the value obtained by reversing the bytes in the specified long value.`

Since:

`1.5`

sum

```
public static long sum(long a,
                       long b)
```

Adds two `long` values together as per the + operator.

Parameters:

`a - the first operand`

`b - the second operand`

Returns:

`the sum of a and b`

Since:

`1.8`

See Also:

`BinaryOperator`

max

```
public static long max(long a,
                       long b)
```

Returns the greater of two `long` values as if by calling `Math.max`.

Parameters:

`a - the first operand`

`b - the second operand`

Returns:

the greater of a and b

Since:

1.8

See Also:

BinaryOperator

min

```
public static long min(long a,
                       long b)
```

Returns the smaller of two `long` values as if by calling `Math.min`.

Parameters:

a - the first operand

b - the second operand

Returns:

the smaller of a and b

Since:

1.8

See Also:

BinaryOperator

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.